

Software Requirement Specification

Courtify

(A Web-Based Sports Arena Booking System)

Prepared by Team Members:

Muhammad Wasiq Shaikh (ID: 09205)
Muhammad Zarrar Ul Arfeen (ID: 08765)
Yousuf Aijaz (ID: 09222)
Shaheer Bin Tariq (ID: 09213)

Prepared for:

Software Engineering (Spring 2026)
Habib University

February 22, 2026

Contents

1	Introduction & Project Overview	2
1.1	Problem Statement	2
1.2	Project Purpose	2
1.3	Scope	2
2	Functional Requirements	3
3	Non-Functional Requirements	3
4	Use Cases/ User Stories	5
5	Out of Scope	6
6	Assumptions, Constraints & Dependencies	7
6.1	Assumptions	7
6.2	Constraints	7
6.3	Dependencies	7
7	Supporting Models & Diagrams	8
7.1	Project Management	9

1 Introduction & Project Overview

1.1 Problem Statement

Currently, the sports ecosystem in Karachi is fragmented. Players struggle to find available courts without physically visiting locations or making multiple phone calls. Conversely, court owners rely on manual methods to track bookings, leading to double-booking errors and inefficient utilization of time slots. There is no centralized platform bridging the gap between sports enthusiasts and arena owners.

Additionally, players have no standardized way to compare arena details such as operating hours, rates, or available sports. Court owners lack any digital dashboard that consolidates bookings, making it difficult to view daily schedules or track court utilization through a single interface.

1.2 Project Purpose

Courtify is a full-stack web-responsive application designed to streamline the discovery, booking, and management of sports courts. Drawing inspiration from platforms like Foodpanda and Airbnb, Courtify aims to digitize the local sports ecosystem, providing players with an intuitive booking interface and owners with robust management tools.

Courtify focuses on essential features only: real-time slot visibility, simple booking flows, and basic listing/availability management for owners. The system intentionally avoids payment processing and advanced analytics to remain feasible within the semester timeline.

1.3 Scope

The system will be a web-based platform supporting three specific user roles:

1. **Players:** Discovery of arenas, checking availability, and booking courts.
2. **Court Owners:** Management of listings, schedules, and analytics.
3. **Admins:** Platform oversight, user moderation, and system integrity.

Only essential CRUD operations for arenas, users, and bookings are included. All bookings will rely on a single-slot model (hour-based), ensuring straightforward validation and preventing complex scheduling conflicts.

The geographic scope is limited to sports arenas within Karachi.

2 Functional Requirements

Req ID	Feature	Description & Acceptance Criteria
Player (General User) Modules		
FR-001	Registration & Auth	The system shall allow users to register via Email/Password. Testable Criteria: User receives a verification email; unverified users cannot book.
FR-002	Arena Search	Users shall be able to filter arenas by location and sport type. Testable Criteria: Search results update within 2 seconds of applying a filter.
FR-003	Booking Management	Users shall view available time slots and book a specific slot. Testable Criteria: System prevents selection of already booked slots (Double Booking Validation). Additions: System shall store booking timestamps for auditing; system shows an on-screen confirmation on success.
Court Owner Modules		
FR-004	Availability Mgmt	Owners shall be able to add or update time slots for their courts. Testable Criteria: Updates allow/disallow player bookings in real-time. Addition: Owners can disable a slot only if it has no active booking.
FR-005	Analytics Dashboard	Owners shall view a dashboard showing upcoming and past booking history. Testable Criteria: Dashboard displays correct count of total bookings for the current month. Clarification: Dashboard will show only basic summaries (totals and upcoming), not advanced graphs.
Admin Modules		
FR-006	Content Moderation	Admins shall be able to remove suspicious courts or ban users. Testable Criteria: Banned user credentials are rejected upon next login attempt. Addition: Admin interface includes a simple search bar to find users or arenas by name or email.

3 Non-Functional Requirements

- **Security (NFR-001):** All user passwords must be hashed using **Bcrypt** before storage in the MySQL database. JWT tokens will be used for session management.

- **Performance (NFR-002):** The React.js frontend must render the "Arena Search" results in under **4 seconds** over a standard 4G connection.
- **Reliability (NFR-003):** The system shall enforce real-time validation to ensure **0% rate of double-bookings** for the same time slot.
- **Usability (NFR-004):** The web interface must be fully responsive, adapting to screen widths from mobile (375px) to desktop (1920px).
- **Maintainability (NFR-005):** The codebase should follow a consistent folder structure for frontend and backend to enable quick updates during the semester.
- **Scalability - Limited (NFR-006):** The system must handle at least **20 simultaneous users** to support classroom demonstrations.

4 Use Cases/ User Stories

Use Case ID	UC-01
Use Case Name	Book a Sports Court
Primary Actor	Player (General User)
Pre-conditions	User is logged in and verified.
Main Flow	1. Player navigates to the search page. 2. Player filters by "Cricket" and selects an Arena. 3. System displays available time slots. 4. Player selects a slot (e.g., 8:00 PM - 9:00 PM) and clicks "Book". 5. System validates availability in real-time. 6. System confirms booking and updates dashboard.
Alternate Flow	5a. Slot was taken by another user milliseconds ago. 5b. System displays "Slot no longer available" error. 5c. System refreshes the availability view.
Post-condition	The selected slot becomes unavailable immediately to all other users.

Table 2: Use Case: Booking Flow

Use Case ID	UC-02
Use Case Name	Add New Arena Listing
Primary Actor	Court Owner
Pre-conditions	Owner is logged in and verified.
Main Flow	1. Owner navigates to the "My Courts" dashboard. 2. Owner clicks "Add New Arena". 3. Owner enters details (Name, Location, Hourly Rate, Sports Type). 4. Owner uploads images and sets operating hours. 5. System validates input and saves the arena to the database. 6. System displays success message and listing goes live.
Alternate Flow	5a. Mandatory fields (e.g., Location) are missing. 5b. System highlights missing fields with error messages. 5c. Owner corrects details and resubmits.
Post-condition	Arena remains hidden from players until an Admin approves it.

Table 3: Use Case: Add New Arena

Use Case ID	UC-03
Use Case Name	Update Slot Availability
Primary Actor	Court Owner
Pre-conditions	Arena exists and Owner is logged in.
Main Flow	1. Owner selects a specific Arena from their dashboard. 2. Owner views the calendar and clicks a specific date. 3. Owner toggles a time slot (e.g., 5:00 PM) from "Available" to "Unavailable" (Maintenance). 4. System updates the slot status in real-time. 5. Slot is immediately hidden from Player search results.
Alternate Flow	3a. The selected slot has an active booking from a Player. 3b. System prevents the update and displays "Active Booking Exists". 3c. Owner must cancel the booking before closing the slot.

Table 4: Use Case: Update Availability

Use Case ID	UC-04
Use Case Name	Ban Suspicious User
Primary Actor	Admin (Moderator)
Pre-conditions	Admin is logged in via secure dashboard.
Main Flow	1. Admin views the "Flagged Activities" report. 2. Admin identifies a user with multiple fake bookings. 3. Admin clicks "Ban User" on the profile. 4. System prompts for confirmation. 5. Admin confirms; System invalidates user session tokens. 6. User is logged out and email is blacklisted.
Alternate Flow	4a. Admin selects "Send Warning" instead of Ban. 4b. System sends an automated warning email to the user. 4c. Case is marked as "Warned" in the logs.

Table 5: Use Case: User Moderation

5 Out of Scope

To ensure feasibility within the Spring 2026 semester, the following features are explicitly excluded:

- **Financial Transactions:** No online payment processing (Stripe/Simpaisa) or automated refunds.
- **Native Mobile Apps:** No iOS (.ipa) or Android (.apk) development; Web-only.
- **Advanced Tech:** No AI-based demand forecasting or dynamic pricing.
- **GPS Tracking:** No real-time GPS location tracking of users.
- **Third-Party Integration:** No integration with external booking platforms.

6 Assumptions, Constraints & Dependencies

6.1 Assumptions

- Court owners will manually honor the bookings made via the web platform (since payments are offline).
- Users will have active email accounts for verification via OTP.
- Arena owners will provide accurate operating hours for their listings.

6.2 Constraints

- **Timeline:** Project must be completed within the Spring 2026 academic semester.
- **Deployment Simplicity:** The system must remain simple enough to run on a single server or local machine for demo.

6.3 Dependencies

- Availability of email at user end for authentication.
- Availability of Internet.

7 Supporting Models & Diagrams

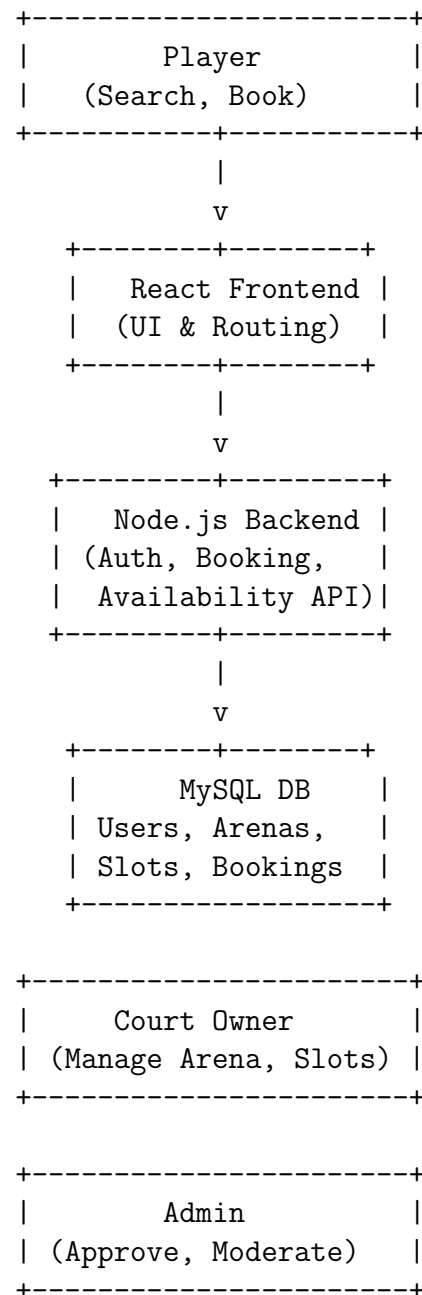


Figure 1: Courtify System Architecture Diagram

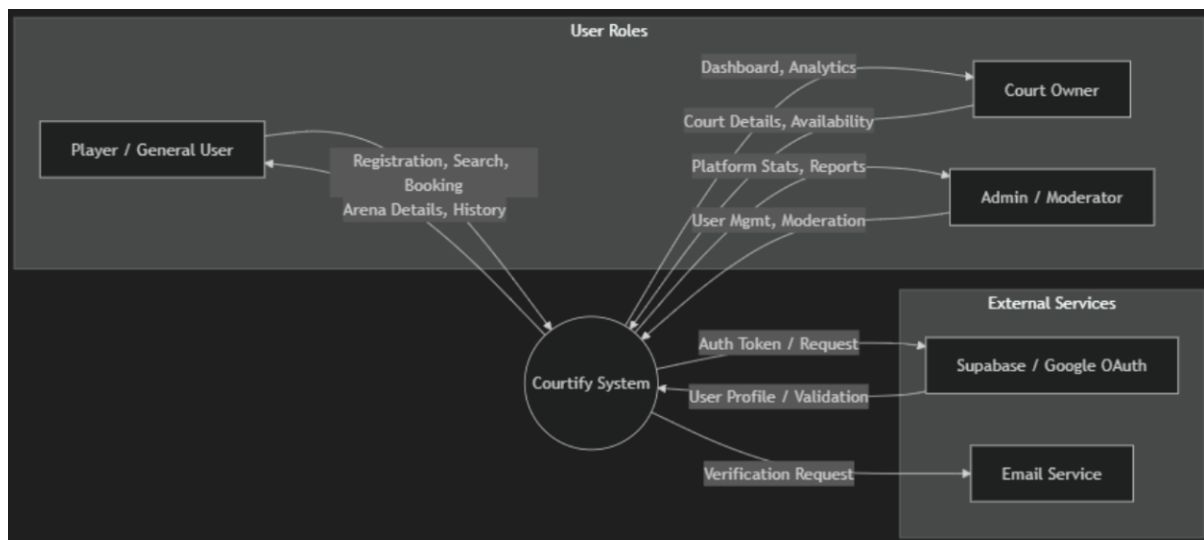


Figure 2: Courtify System Architecture Diagram

7.1 Project Management

We are using Trello to track user stories, tasks, and the development roadmap. You can view our active board using the link below:

[Click Here to View the Courtify Trello Board](#)